

SCL — Bilan de la phase « Modules & Orchestrateur »

Etienne Lamy

2026-07-21

Table des matières

0. Résumé	2
1. Cadre : ce que la phase devait prouver	2
2. Partie I — Le module détecteur/générateur	2
2.1 Le principe : le goulot est le siège de la valeur	2
2.2 Ce qui a marché	3
2.3 Ce qui a échoué — et le signal objectif qui l’a trahi	3
2.4 Le grand échec de la phase : le latent opaque (§5bis) et sa correction	4
3. Partie II — Régimes, hiérarchie, spécialisation	4
3.1 Un régime = une signature compressée récurrente	4
3.2 Verrouillage asymétrique : condition <i>nécessaire</i> de la spécialisation	5
3.3 Naissance : surprise confirmée + grâce ; le problème de sur-crétion	5
3.4 Localiser l’échec dans la hiérarchie	5
4. Partie III — L’orchestrateur	5
4.1 Mode A — recherche typée par la valeur	5
4.2 Mode B — de l’imitation au renforcement (l’orchestrateur-LLM)	5
4.3 Mémoire épisodique & nuit	6
4.4 Auto-réglage réversible (§28.4)	6
5. Partie IV — Les principes chèrement acquis (transversaux)	7
6. Ce qui reste ouvert (limites honnêtes)	7
7. Table de synthèse — étapes 1→15 (chiffres mesurés)	8

Ce qui a marché, ce qui a échoué, et ce qu’on en a appris. Document de phase, versionné (.md canonique, .tex + .pdf dérivés). Rédigé le 2026-07-21. Couvre les étapes 1→15 (POC 2D « sucres/bâtons »). Docs de fond : SCL - Vision et Strategie.md, SCL_fondements_mathematiques.md, Architecture SCL Code v2.md, STATUS.md.

0. Résumé

Cette phase a construit et éprouvé les deux briques centrales de SCL sur un bac à sable 2D : le **module détecteur/générateur** (qui compresse puis régénère un signal, et dont le goulot est le siège de la parcimonie) et l'**orchestrateur** (qui compose ces modules en programmes). Le but n'était pas de « bien jouer » au monde mais de **prouver l'émergence** : catégories, régimes, règles et compositions apparaissent par la mesure, sans être donnés.

Le résultat tient en une phrase : **tout ce qui compte se laisse mesurer sans unité et sans étiquette** — un rappel d'objets dans $[0,1]$, un gain de prédictibilité contre un prior trivial, une familiarité — et c'est cette scale-freeness qui permet à la structure d'émerger et à l'orchestrateur de choisir. Chaque échec de la phase vient, à l'inverse, d'une grandeur **sans référence** (une perte scalaire, un latent opaque, un critère relatif confondu par l'amplitude) ; chaque correctif consiste à **réancrer la mesure**.

Ce document insiste autant sur les impasses que sur les réussites : les impasses sont la matière première de l'auto-diagnostic futur de l'orchestrateur (§28.3 de l'Architecture).

1. Cadre : ce que la phase devait prouver

Le monde 2D (grille, sucres, bâtons, un agent qui accélère, parfois du vent) est un **banc de preuve d'émergence**, pas une tâche à maximiser. On y vérifie que :

1. un module apprend à **compresser puis reconstruire** la vision, avec un **goulot** qui réduit réellement l'information (parcimonie / MDL, §5) ;
2. des **catégories** apparaissent sans classes données ;
3. des **régimes** (vitesses, vent) sont identifiés comme des signatures récurrentes ;
4. une **hiérarchie** se forme (champ $\rightarrow$ régime $\rightarrow$ règle action $\rightarrow$ régime) ;
5. un **orchestrateur** compose les modules en **programmes** et choisit le meilleur par la mesure, d'abord par recherche, puis par une politique **apprise** ;
6. un cycle **jour/nuît** capture l'imprévu le jour et le **comprend** la nuit.

Contraintes non négociables rappelées tout au long : **codage en dur minimal** (toute adaptation manuelle = dette documentée), **parcimonie comme moteur**, **catégories émergentes**, **l'orchestrateur compose des fonctions** (jamais de combinaison linéaire), **verrouillage asymétrique** (plancher jamais plafond), **création sur surprise confirmée**, **apprentissage local**, **activation creuse**.

2. Partie I — Le module détecteur/générateur

2.1 Le principe : le goulot est le siège de la valeur

Un module = un encodeur (champ $\rightarrow$ latent) + un générateur (latent $\rightarrow$ champ). La **taille interne** peut être grande (un bon générateur coûte cher) ; **seul le goulot compte** : il doit produire une sortie **plus petite que l'entrée**. C'est la traduction opératoire du MDL : ce qui survit est ce qui **réduit** la description tout en la **régénérant**.

Mesuré (étape 1) : goulot **64 < 100** pixels, reconstruction des objets à **F1 90 %**. La compression est réelle *et* fidèle.

2.2 Ce qui a marché

- **Convolution avant le goulot**. C’est le biais inductif qui rend la vision compressible ; sans lui, aucune taille de latent ne suffit (voir 2.3).
- **Perte pondérée / classification par cellule**. Le champ est ~90 % vide ; une MSE se minimise en sortant du noir. Pondérer la classe rare (objets) évite l’effondrement.
- **Prédiction T-1 → T comme sonde de vitesse (étape 2a)**. La fidélité de transition est élevée à la **vitesse entraînée** (rappel **84 %**) et s’effondre ailleurs (~**20 %**) : la même mesure sert de **détecteur de régime**, sans détecteur dédié.
- **Catégorisation émergente par VQ (étape 5)**. Un encodeur **1×1** (apparence locale, sans contexte) + quantification vectorielle fait **émerger 4 catégories** (sur 6 prototypes, le reste élagué), **100 % pures**, reconstruction 100 %. Aucune étiquette.
- **Décomposition en objets par slot-attention (étape 4)**. Reconstruction **94 %**, et surtout une **prédiction triviale** : décaler les objets prédit l’image suivante à **80 % sans aucun réseau de prédiction**.
- **Choix de taille par MDL (étape 3)**. Sur un catalogue [8..96], l’orchestrateur naïf choisit **dim 48** — ni sous- ni sur-dimensionné — par le seul compromis gain/coût.

2.3 Ce qui a échoué — et le signal objectif qui l’a trahi

Échec	Symptôme mesuré	Correctif générique
Effondrement à zéro	perte basse <i>mais</i> rappel objets nul	perte pondérée ; ne jamais conclure sur une perte scalaire
Plafond insensible à la capacité	MLP à ~44 % que le latent fasse 48, 100 ou 200	ce n’est pas la taille, c’est le biais inductif → conv ; balayage de capacité comme diagnostic
Compression fidélité pure	conv pleine résolution reconstruit à 100 % mais expanse (300 > 100)	le goulot doit réduire , pas seulement copier
Critères sans unité	latent brut à magnitudes arbitraires (résidus 50-60, seuils absurdes)	normaliser + exprimer tout critère en ratio au prior trivial
Statistiques à queue lourde	moyenne médiane ; un compteur de pas ne déclenche jamais	borner + lisser (EMA) avant de décider
Slots naïfs	21 % — les slots ne se spécialisent pas	compétition explicite (softmax sur les slots) + itération → 94 %
Champ réceptif trop large	catégories « sales »	encodeur 1×1 (apparence locale) → catégories pures
Batch-1 en ligne	ne converge pas sur entrées quasi-aléatoires	mémoire de jeu + mini-lot (rester en ligne, gagner en stabilité)

Chacun de ces échecs partage la même racine : **une grandeur qu'on croyait informative ne l'était pas**, faute de référence. Le diagnostic tient dans le fait de toujours disposer d'un **prior trivial** contre lequel se juger.

2.4 Le grand échec de la phase : le latent opaque (§5bis) et sa correction

Toute la hiérarchie initiale (étapes 6→9) était bâtie sur le **latent compressé opaque** du module de vision. Les mesures ont fini par désigner ce maillon comme le **goulot d'étranglement du projet** :

représentation	qualité de prédiction
latent compressé opaque (64) — <i>utilisé par la hiérarchie</i>	57 %
latent spatial (non compressant)	80 %
représentation objet (slot-attention)	94 % recons. / 80 % préd. triviale

Conséquence en cascade : des modules-régime qui prédisent mal ne voient leur erreur **que faiblement** dégradée par un changement de régime → **la nouveauté devient indétectable**. C'est la cause profonde de l'échec de l'étape 9 : un **vent transverse** ne déclenchait **aucune naissance**, et pire, la *familiarité montait* quand le vent se levait (cf. enseignement #11).

Correction (étape 10) — travailler en espace-champ. Les modules-régime prédisent désormais **champ(T-1) → champ(T)** directement. Le résidu devient le **rappel d'objets [0,1] : sans unité par construction**, sans normalisation ni ratio. Un vent en Y fait rater la prédiction (qui décale en X) → le rappel chute → la nouveauté est **immédiatement visible**.

Mesuré (étape 10) : détection nette (**56-59 %** sur le bon régime vs **20-27 %** ailleurs), 3 régimes sur 3 couverts, et **vent transverse enfin détecté** (familiarité **59 → 28 %**, un module naît) — ce que le latent opaque ne faisait jamais.

Leçon centrale. Le bon espace de représentation n'est pas le plus compressé, c'est celui où **la mesure de surprise est fidèle**. Un rappel d'objets est scale-free ; un latent opaque ne l'est pas. La compression sert la reconstruction ; elle ne doit pas être imposée à la couche qui doit **détecter le changement**.

3. Partie II — Régimes, hiérarchie, spécialisation

3.1 Un régime = une signature compressée récurrente

Un module-régime n'encode pas « la vitesse (1,0) » ; il encode **une manière dont le champ se transforme** qui revient assez souvent pour valoir un module. C'est plus générique qu'une étiquette de vitesse : un vent, un frottement, tout motif de transformation récurrent est éligible.

Mesuré (étape 6) : 4 modules nés, **3/3 régimes couverts**, les niveaux N1 (champ) et N2 (identité de régime) **concordent**. **(étape 7) :** N3 apprend la règle **action → changement de régime** (l'accélération) avec exactitude **57 % vs 38 % trivial (+31 %)**, et **(étape 8)** un horizon de prédiction **naturel à T+5** émerge de la courbe G(h) — mesuré, pas choisi.

3.2 Verrouillage asymétrique : condition *nécessaire* de la spécialisation

Sans verrou, un module compétent **se ré-entraîne sur le régime suivant et oublie le sien** : couverture 2/3. Avec un verrou **asymétrique** (on fige le plancher de compétence, jamais un plafond), la spécialisation tient : **3/3**. C'est un résultat empirique fort : la mémoire longue d'un module vient d'un **arrêt d'apprentissage local**, pas d'une capacité plus grande.

3.3 Naissance : surprise confirmée + grâce ; le problème de sur-crétion

Créer sur le moindre incident donne **799 modules en 800 pas**. Deux garde-fous : - **surprise confirmée** (leaky-evidence / EMA) : un pic isolé ne suffit pas ; - **grâce** : un nouveau-né garde la main le temps d'apprendre, sinon on en crée un autre avant qu'il ait fini — c'est la **sur-crétion**.

Les modules conv champ→champ sont **lents** (~2000 pas). Un **progress-gate** (« ne pas créer tant que le meilleur module progresse encore », §28.1) réduit la casse, mais un résidu de sur-crétion est resté (jusqu'à 5 modules pour 3 régimes). **Il sera résolu par l'auto-réglage** (§3.4 / étape 15).

3.4 Localiser l'échec dans la hiérarchie

La qualité d'un niveau **plafonne le niveau au-dessus**. À l'étape 7, les erreurs de N3 se localisent **au niveau N2** : le vocabulaire de régimes confond $v=(1,0)$ et $v=(2,0)$ dans un même module. Règle (§29.4) : **diagnostiquer le niveau le plus bas anormal** avant de toucher au niveau supérieur. Corollaire méthodologique éprouvé (enseignement #12) : vérifier qu'un stimulus « nouveau » l'est **vraiment** — un vent (1,0) sur une vitesse (1,0) produit un déplacement (2,0) *déjà appris* ; le système avait raison de le reconnaître, c'était l'expérience qui était mal conçue.

4. Partie III — L'orchestrateur

L'orchestrateur ne calcule jamais lui-même : il **compose des modules en programmes**. Un programme enchaîne des **opérateurs typés** (compresser : champ→latent, generer : latent→champ, predire_champ : champ→champ, predire_latent : latent→latent) et transforme un signal de départ en une **cible ancrée** (le vrai champ suivant). Le **typage** rend absurdes-impossibles la plupart des programmes : il élague l'espace de recherche *avant* toute mesure.

4.1 Mode A — recherche typée par la valeur

Mode A énumère les programmes bien typés, **entraîne chacun**, mesure son **gain de prédictibilité** $G = 1 - \text{résidu}(\text{module}) / \text{résidu}(\text{prior trivial})$, et retient le meilleur au sens **valeur = G - coût** (le coût = somme des goulots mobilisés).

Mesuré (étape 11) : sans aucune préférence câblée, Mode A **choisit predire_champ** ($G = 64\%$), **rejette** la chaîne latent-opaque (36 %) et la reconstruction pure (2 %). Il **redécouvre §5bis tout seul** : le champ direct bat le latent opaque, par la mesure.

4.2 Mode B — de l'imitation au renforcement (l'orchestrateur-LLM)

Mode B est l'orchestrateur **pensé comme un LLM à attention** : au lieu de *chercher*, il **émet** un programme token par token, **conditionné par l'objectif**, sous **masque de type** (à chaque pas, seules les continuations bien typées sont émettables). Deux voies, toutes deux validées :

(a) **Imitation (étape 13).** Mode B distille les programmes gagnants de Mode A. Résultat clé : le meilleur programme **dépend de l'objectif** — prédire → **predire_champ** ; reconstruire → **compresser** → **generer** — et Mode B **émet le bon sans refaire la recherche**. Il **amortit** le coût de Mode A.

(b) **Renforcement, sans professeur (étape 14).** À partir de la **seule récompense** $R = G - \text{coût}$, init aléatoire, Mode B **découvre** les deux optima (2/2). Table de récompense mesurée :

programme	R (prédiction)	R (reconstruction)
predire_champ	+0.459	+0.161
compresser → generer	-0.080	+0.628
predire_champ → compresser → generer	+0.304	+0.073
compresser → generer → predire_champ	+0.291	+0.080
compresser → predire_latent → generer	+0.103	+0.095

Les objectifs **s'opposent** : **predire_champ** est le meilleur en prédiction et quasi-pire en reconstruction ; **compresser** → **generer** l'inverse. La dépendance au contexte est réelle, et Mode B l'apprend.

L'échec, et l'évolution qui l'a résolu. Le REINFORCE nu **échouait** sur la reconstruction : la politique **partagée** s'effondrait sur le programme le plus **court** (**predire_champ**, un seul token, donc le plus facile à échantillonner) — un **local-optimum honnête** de la reconstruction ($R = +0.16$) — et **n'échantillonnait jamais** le vrai gagnant à 2 tokens ($R = +0.63$). L'avantage positif n'arrivait donc jamais. Correctif générique : **régularisation par entropie à recuit** — on maintient l'exploration au début (l'entropie des distributions de pas est ajoutée à l'objectif), puis on laisse **exploiter** en fin. Avec elle : 2/2.

Leçon. Une politique apprise sur récompense a un biais vers les programmes **courts** (faciles à tirer) ; sans pression d'exploration explicite, elle rate les compositions plus longues mais meilleures. L'entropie recuite est le pendant, côté orchestrateur, de la « grâce » côté modules : on protège l'exploration le temps qu'un candidat prometteur se révèle.

4.3 Mémoire épisodique & nuit

- **Jour** : on ne mémorise que le **non-régénérable** — la *graine* d'un épisode (champ initial + actions + modules actifs + résidu), pas les images. On ne capture que le **surprenant**. Une **hystérésis** obligatoire (entrer en surprise sous un seuil bas, en sortir au-dessus d'un seuil haut) évite de **fragmenter** un même épisode.
- **Nuit** : on **rejoue** l'épisode et on entraîne un module dédié jusqu'à le **prédire**. « Compris » = *régénérable ET prédit*, critère mesurable.

Mesuré (étape 12) : un vent capturé le jour (5 épisodes cohérents, familiarité ~25 %) est **entièrement compris la nuit** (rappel de jeu au-dessus du seuil).

4.4 Auto-réglage réversible (§28.4)

Dernier maillon : la boucle qui règle les **hyperparamètres de l'orchestrateur sans intervention humaine**, de façon **réversible** — on ne garde un changement que s'il améliore un **observable mesuré**, au-delà d'une marge (asymétrie : on ne court pas après le bruit) ; sinon on **revient en arrière**. Le cœur est **générique** (ni monde ni torch) : on lui injecte `appliquer(valeur)` et `mesurer()` → `score`.

Mesuré (étape 15) — branché sur `grace_regime` contre le résidu de sur-cr  ation,   horizon `pas=3000` :

gr��ce	modules	couverture	observable (couv. — · superflus)
1000	9	52 %	−0.09
2000	5	48 %	+0.29
4000	3 (= l'id��al)	44 %	+0.44 (gard��)

La boucle **r  sout la sur-cr  ation** ($5 \rightarrow 3$ modules, un par r  gime) par la seule mesure, sans qu'on lui donne la valeur.

Insight (dette). La gr  ce optimale **d  pend de l'horizon d'entra  nement** (1000   `pas=1000`, 4000   `pas=3000`) : une gr  ce exprim  e en **pas absolus** est intrins  quement li  e   la dur  e. Le crit  re de naissance gagnerait   compter en **progr  s** (d  j   via `_progresse`) plut  t qu'en pas fixes, ou   **indexer la gr  ce sur l'horizon**.

5. Partie IV — Les principes ch  rement acquis (transversaux)

1. **Ne jamais conclure sur une perte scalaire.** Suivre le rappel de la classe rare ; une perte basse peut cacher un effondrement.
2. **Toute d  cision se prend sur une grandeur scale-free.** Rappel $[0,1]$, gain contre un prior trivial, familiarit  . Les magnitudes brutes trompent.
3. **Le biais inductif prime sur la taille.** Convolution avant le goulot ; 1×1 pour des cat  gories pures. Un balayage de capacit   *diagnostique* le biais manquant.
4. **La compression sert la reconstruction, pas la d  tection.** D  tecter le changement demande un espace o   la surprise est fid  le (espace-champ), pas le plus compress  .
5. **Verrouillage asym  trique = m  moire.** La sp  cialisation vient d'un arr  t d'apprentissage local, pas d'une capacit   accrue.
6. **Cr  er sur surprise confirm  e + gr  ce.** Prot  ger le nouveau-n   le temps qu'il apprenne ; sinon sur-cr  ation.
7. **Prot  ger l'exploration.** C  t   orchestrateur, l'entropie recuite emp  che l'effondrement sur les solutions faciles ; c'est la « gr  ce » des programmes.
8. **Rendre les r  glages r  versibles.** Aucun hyperparam  tre n'est sacr   : on le garde s'il am  liore un observable, sinon on revient.
9. **Diagnostiquer le niveau le plus bas anormal** avant de toucher au-dessus.
10. **V  rifier qu'un stimulus nouveau l'est vraiment** avant d'accuser le syst  me.

Ces dix principes sont exactement la **mat  re de l'auto-diagnostic** de l'orchestrateur (Architecture §28.3) : chaque sympt  me a un signal objectif et un correctif g  n  rique.

6. Ce qui reste ouvert (limites honn  tes)

- **Rappels conv mod  r  s** (~44-59 % en run court) : modules sous-entra  n  s ; ce n'est pas un plafond de principe mais un budget d'entra  nement.

- **Bascule mesurée A → B** : *quand* cesser de chercher (Mode A) pour émettre (Mode B) — décision méta non encore implémentée.
- **Instrumentation** des nouveaux objets (programmes, épisodes, réglages) dans le viewer.
- **L'action n'est pas encore intégrée** au déroulé de l'orchestrateur : c'est l'objet de la **phase suivante** (objectifs faim/curiosité, arbre d'actions exponentiel, A* avec g()/h() émergés, renforcement nocturne de plus en plus amont).

7. Table de synthèse — étapes 1→15 (chiffres mesurés)

#	Capacité	Résultat mesuré
1	Compresser & reconstruire	goulot 64 < 100 , F1 90 %
2a	Prédire T-1→T = sonde de vitesse	84 % à la vitesse entraînée vs ~20 % ailleurs
2b	Chaîne module→module	80 % (latent spatial) / 57 % (latent opaque)
3	Taille par MDL	choisit dim 48 sur [8..96]
4	Attention/objets → prédiction triviale	recons. 94 %, prédiction en décalant 80 %
5	Catégorisation émergente	4 catégories pures à 100 %, aucune étiquette
6	Détection de vitesse	4 modules, 3/3 régimes , niveaux concordants
7	Règle action → régime (N3)	57 % vs 38 % trivial (+ 31 %)
8	Horizons	horizon naturel T+5 (mesuré)
9	Vent : localiser l'échec	signature obtenue mais aucune naissance → corrigé en 10
10	Régime en espace-champ (lève §5bis)	56-59 % vs 20-27 %, vent détecté (59→28 %)
11	Orchestrateur Mode A	choisit predire_champ (G 64 %), redécouvre §5bis
12	Boucle jour→nuit	vent capturé le jour, compris la nuit
13	Mode B imitation	émet le bon programme par objectif , sans recherche
14	Mode B renforcement	découvre les 2 optima (2/2) via entropie recuite
15	Auto-réglage §28.4	grace=4000 → 3 modules : sur-création résolue par la mesure

Reproductible : chaque ligne a un harnais `python3 -m scl.etapeN_...` (voir `STATUS.md`). Suite complète : 223 tests verts.